

AI PM Eval Cheat Sheet

Build evals from scratch for any AI product — from the AI PM Eval Masterclass v3

THE 5 QUESTIONS

Answer in order. Each depends on the last. Skip one and the chain breaks.

Q1: DO → Q2: FAIL → Q3: SUCCESS →

Q4: DETECT → Q5: IMPROVE

1 What does the agent DO?

Break the product into discrete jobs. Map every task. Each job gets its own eval.

2 How can it FAIL at each job?

For each task, enumerate specific failure modes. Read 30-50 real traces. Open-code without categories first.

3 What does SUCCESS look like?

Define rubrics. Turn "good" into a number. Write a Good Output Document with floor & ceiling.

4 How do I DETECT failure automatically?

Structural checks, LLM-as-judge, human review. Every change triggers the suite.

5 How do I IMPROVE the product?

Detect → diagnose root cause → fix → verify with evals → ship → add to regression suite.

ACCT FRAMEWORK

Every quality dimension maps to one of these four. If a criterion spans multiple, split it.

A

Accuracy

Are the facts correct? Do claims trace to sources? No hallucinations, no fabricated data.

C

Completeness

Did it cover everything needed? No missing steps, no skipped requirements, no partial answers.

C

Calibration

Does confidence match evidence? Hedges when uncertain, confident when supported. No overconfidence.

T

Taste

Tone, style, brand voice, specificity, creativity. The subjective "would I send this to a client?" dimension.

7 UNIVERSAL FAILURE MODES

Checklist against your product. Not all apply — focus on what appears in your traces.

#	FAILURE MODE	WHAT HAPPENS	QUICK TEST
1	Generic Trap	Output is vague, applies to anyone	Swap brand name — still works?
2	Sycophancy	Agrees with everything, never pushes back	State something wrong — does it validate?
3	Hallucination	8 real facts + 2 invented, blended seamlessly	Fact-check each claim individually
4	Format Break	Good content, wrong structure	Parse output programmatically
5	Instruction Drift	Follows 8/10 instructions, silently ignores 2	Checklist each instruction independently
6	Overconfidence	States uncertain things as facts	Check language vs. source certainty
7	Stale Info	Uses outdated data as current	Check dates in cited facts

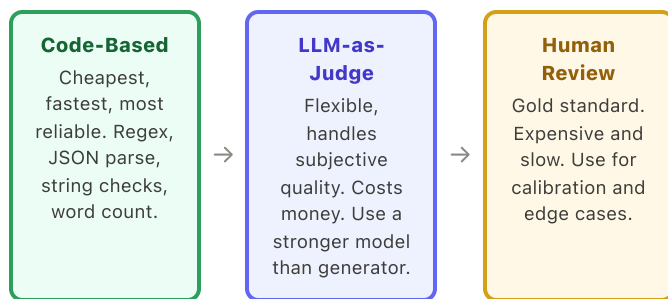
5-POINT QUALITY SCALE

Define what each score means for YOUR product. 3 is the quality floor.

- 1 Unusable.** Fundamentally wrong or harmful. Would damage trust.
- 2 Poor.** Some value but errors too severe. User starts over.
- 3 Acceptable.** Quality floor. Gets the job done. Needs editing, not redoing.
- 4 Good.** Correct, complete, appropriate. Minor polish only.
- 5 Exceptional.** Quality ceiling. Insightful, synthesized. Use as-is.

EVAL HIERARCHY

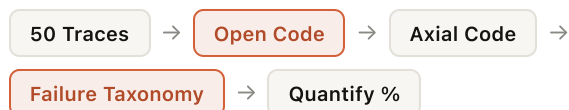
Always prefer the simplest method that works. Move right only when needed.



FIRST WEEK PLAYBOOK

DAY	ACTIVITY	DELIVERABLE
1-2	Read 30 production traces. Freeform notes. No categories yet.	30 annotated traces
3	Shadow 10 user sessions. Note expected vs. actual behavior.	Gap observations
4	Interview engineering team. Map their mental model.	Blueprint alignment
5	Write State of Quality doc. Share with team.	Quality baseline

ERROR ANALYSIS PROCESS

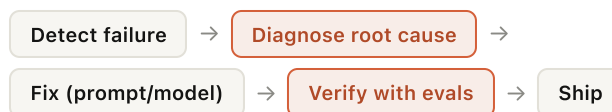


- **Open coding:** Write freeform labels for every issue in each trace. No predefined categories.
- **Axial coding:** Feed open codes into a frontier LLM. Ask it to propose 5-6 failure categories.
- **Quantify:** What % of failures fall in each category? Focus evals on the top 3.
- **Formalize:** Turn top categories into pass/fail criteria with 3-5 example traces each.

BUILD YOUR FIRST EVAL

1. **Pick ONE job** — highest user-facing impact from your task map.
2. **Pick ONE dimension** — most common failure mode from your error analysis.
3. **Write 20 test cases** — real inputs from production, not synthetic. Include 5 known failures.
4. **Define the rubric** — 1-5 scale with concrete descriptions at each level for this dimension.
5. **Choose your grader** — code check if possible, LLM judge if subjective.
6. **Run 3x per test case** — AI is non-deterministic. Single runs lie. Take the median.
7. **Baseline score** — this is your starting point. Every change is measured against it.

THE IMPROVEMENT LOOP



Then add the failure to your regression suite so it never comes back. +3 test cases/week = 156/year. This compounds into a quality moat.

10 COMMANDMENTS

1. **Look at data first, build evals second.** 60-80% of time = error analysis.
2. **You own the rubric.** Engineers own infrastructure. Don't delegate "what good looks like."
3. **Start with 20 test cases, not 200.** Well-chosen real failures beat large synthetic sets.
4. **Prefer code checks over LLM judges.** Use the simplest method that works.
5. **Run each test case multiple times.** Single runs show what CAN happen. Multiple show what WILL.
6. **Never use the same model as generator and judge.** Self-enhancement bias inflates scores.
7. **Report distributions, not just averages.** Avg 4.0 with catastrophic 1s is worse than consistent 3.5.
8. **Check statistical significance.** 0.3-point gain on 15 cases might be noise.
9. **Add every production failure to your test suite.** Regression cases = "never again" insurance.
10. **Calibrate your judge quarterly.** Grade 30 outputs yourself. Compare. Kappa < 0.6 = rewrite.

GOODHART'S WARNING

"When a measure becomes a target, it ceases to be a good measure." Use multiple dimensions (ACCT). One number is easy to game. Five dimensions force real improvement.

THE #1 TRAP: PREMATURE AUTOMATION (Q4 BEFORE Q1-Q3)

Don't buy an eval platform and write 200 test cases on day one. Without error analysis (Q2) and rubrics (Q3), you'll measure the wrong things with expensive precision. Do the cheap thinking first.